

Relatório 6 - Prática: FastAPI (II)

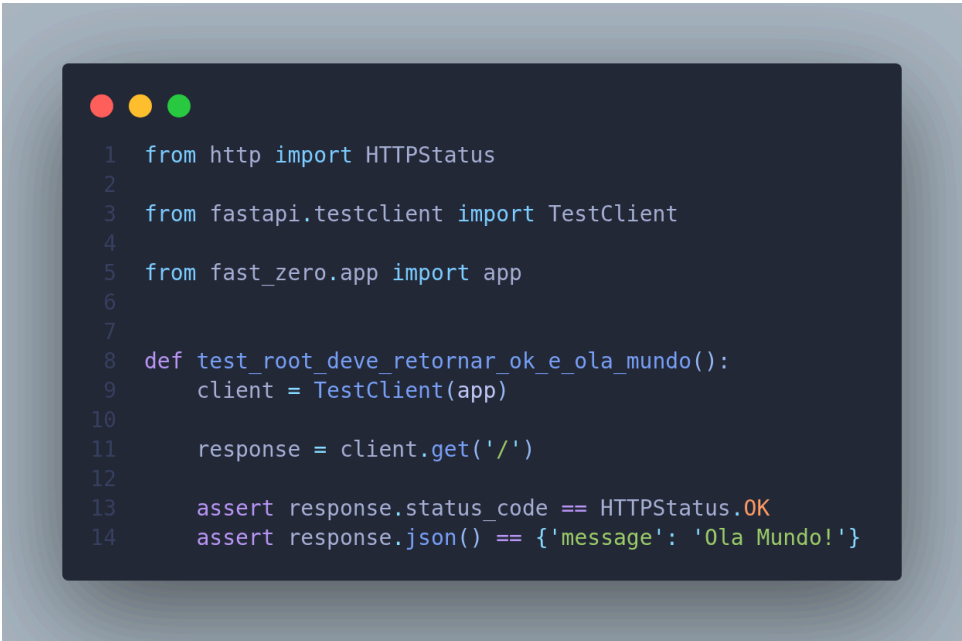
José Carlos Seben de Souza Leite

Descrição da atividade

Começando agora com FastAPI nesse card iniciamos com o básico, instalando o pipx para podermos estar instalando bibliotecas (**como nesse caso foi com a FastAPI**) isoladamente de forma global, o **poetry** para gerenciar nosso projeto com Python e cuidar das versões.

Além dessas também baixamos o próprio FastAPI, e algumas ferramentas para nos auxiliar no desenvolvimento do projeto como a **Taskipy** para como o nome já indica utilizamos para escrever testes para nossa API, e algumas outras ferramentas como o **Ruff** que serve para analisar o nosso código e verificar se estamos seguindo as boas práticas do Python, e por último duas ferramentas uma sendo a **Typos** que vamos utilizar para verificar erros ortográficos em escritas em inglês, e a outra sendo a **Taskipy** para definirmos comando personalizados para operar no terminal.

O que criamos até agora referente a códigos foi apenas um “Hello Word” e criamos um teste para o mesmo, utilizando a Taskpy:



```
1 from http import HTTPStatus
2
3 from fastapi.testclient import TestClient
4
5 from fast_zero.app import app
6
7
8 def test_root_deve_retornar_ok_e_ola_mundo():
9     client = TestClient(app)
10
11     response = client.get('/')
12
13     assert response.status_code == HTTPStatus.OK
14     assert response.json() == {'message': 'Ola Mundo!'}
```

Um teste simples apenas importando as dependências necessárias da ferramenta e o que vamos testar, simplesmente o que ele faz é funcionar como um cliente utilizando **TestClient** e fazendo uma requisição em '/' que é o caminho onde está nosso hello word, finalizando se o teste for bem-sucedido retornando um OK e a mensagem de Ola Mundo!.

Finalizando esta parte do card acabamos um o quiz sobre os conteúdos vistos até agora.

Progresso do quiz

Respondido: 10 / 10 (100%)



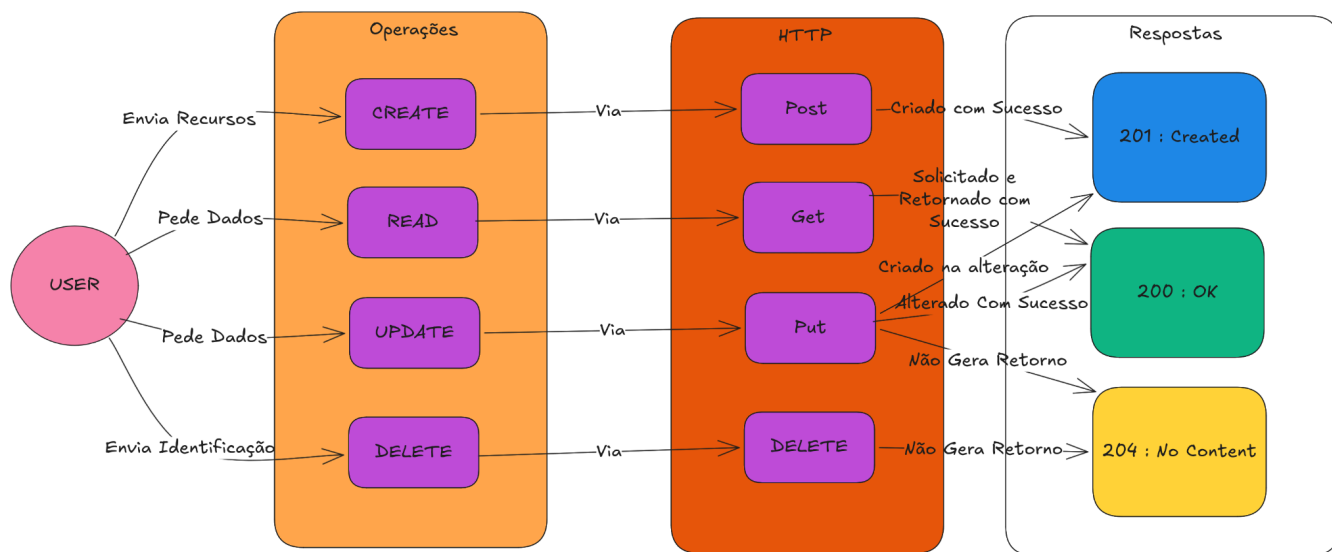
Correto: 10 / 10 (100%) [Reiniciar](#)

Seguindo no card entramos no conceito de **CRUD** (create, read, update, delete) basicamente interações que realizamos com o banco de dados.

- **Create:** Criar dados
- **Read:** Ler ou recuperar dados.
- **Update:** Atualizar dados já existentes.
- **Delete:** Deletar dados.

Quando trabalhamos com protocolos como, por exemplo, o protocolo HTTP e APIs temos operações semelhantes com nomenclaturas diferentes.

- **Post:** Requisição realizada pelo cliente para que o servidor aceite um novo dado.
- **Get:** Requisição realizada pelo cliente ao servidor pedindo um dado.
- **Put:** Requisição para atualização de algum dado.
- **Delete:** Requisição para que o servidor delete algum dado.



Para desenvolvermos nosso “CRUD” fizemos apenas uma simulação de um banco de dados falso em um array **database = []** utilizando apenas dados salvos temporariamente como estamos fazendo uma simulação de um CRUD atendeu nossa necessidade.

No desenvolvimento do código durante a aula fizemos praticamente um exemplo de cada parte **post, get, put e delete**. E um teste de cada um deles para verificarmos o funcionamento do projeto. Um adendo sobre os testes que escrevemos ele está em um formato totalmente errado por serem testes sequencias onde um esta dependendo do outro como em um teste cria o usuário e no outro teste usa o usuário criado no teste anterior para executar, mas como foi só para aprendizado e de uma simulação **no curso** foi mantido assim.

Exemplos de operação realizado no curso:

Post

```
1 @app.post('/users/', status_code=HTTPStatus.CREATED, response_model=UserPublic)
2 def create_user(user: UserSchema):
3     user_with_id = UserDB(**user.model_dump(), id=len(database) + 1)
4
5     database.append(user_with_id)
6
7     return user_with_id
8
```

Esta função utilizando o post para criar um novo usuário, funcionando com um retorno de uma mensagem de usuário criado com CREATED que seria a saída 201 e um schema de usuário que criamos sendo o UserPublic, criado para termos uma saída com o nome de usuário, email, e id, mas não mostrar a senha do mesmo, a função cria um novo usuário em UserSchema e depois atribui um id para o mesmo fazendo a conta do tamanho do banco de dados mais um, assim o primeiro ficando com o id um e assim por diante.

Get

```
1 @app.get('/users/', status_code=HTTPStatus.OK, response_model=UserList)
2 def read_users():
3     return {'users': database}
```

O get foi utilizado apenas duas vezes uma para fazermos nosso print do hello word e outra aqui para fazermos uma função que utilizamos para verificar no nosso banco de dados os usuários salvos, ou seja, uma função para nos retornar nossa lista de perfis criados.

Put

```
1 @app.put(  
2     '/users/{user_id}', status_code=HTTPStatus.OK, response_model=UserPublic  
3 )  
4 def update_user(user_id: int, user: UserSchema):  
5     user_with_id = UserDB(**user.model_dump(), id=user_id)  
6  
7     if user_id > 1 or user_id > len(database):  
8         raise HTTPException(  
9             status_code=HTTPStatus.NOT_FOUND, detail='User not found!'  
10        )  
11  
12     database[user_id - 1] = user_with_id  
13     return user_with_id  
14
```

Função que utilizamos para atualizar os dados de algum usuário já cadastrado utilizando o id para localizar um usuário em específico e no caso desse mesmo não existir ou o número de id pesquisado for maior que o tamanho do nosso banco de dados retornar o erro 404 e uma mensagem de usuário não encontrado, quando o usuário é encontrado podemos mudar seus dados exceto o id que vai se manter o mesmo sempre o mesmo retornando no formato de UserPublic sem mostrar a senha, e uma mensagem de sucesso 200.

Delete

```
1 @app.delete(  
2     '/users/{user_id}', status_code=HTTPStatus.OK, response_model=UserPublic  
3 )  
4 def delete_user(user_id: int):  
5     if user_id > 1 or user_id > len(database):  
6         raise HTTPException(  
7             status_code=HTTPStatus.NOT_FOUND, detail='User not found!'  
8        )  
9     return database.pop(user_id - 1)  
10
```

Nossa função de delete usa uma verificação assim como a do put para encontrar se o usuário existe ou não e o que ela faz de diferente é a remoção do perfil do usuário selecionado do banco de dados e o retorno dos dados do perfil removido no modo de UserPublic para visualizarmos quem foi removido e a confirmação com o código 200.

Já sobre os testes escritos nesse card não tem muito o que falar é uma bancada de testes sequencias onde um depende do outro e passa por essas nossas funções que detalhei a cima indo deis da criação de um novo usuário no banco, uma pesquisa, depois uma alteração no usuário criado e por fim a remoção desse mesmo usuário.

Para o código prático por este ser o primeiro card de back-end e não ter sido passado nada muito mirabolante para criação de um sistema novo para teste, resolvi pegar o código que já temos da aula e fazer algumas melhorias no mesmo.

Ajustamos os testes que como no próprio curso é falado que aquela maneira é completamente errada os testes dependendo um do outro e a maneira correta e comercial é cada teste ser independente e conseguir passar por as três “etapas” sem depender dos demais, cada teste deve Arrange (preparar), Act (agir) e Assert (verificar) de forma completamente independente foi isso que foi feito.

Agora antes de cada teste limpamos a database para não ficar dados persistindo entre testes que poderia causar problemas, também adicionamos as regras para criação de um cliente nas configurações de teste permitindo que quando o teste precisar de um cliente utilizar aquele que está nas configurações assim antes de executar o próximo teste limpamos o banco de dados, assim mesmo se o cliente que está nas configurações for alterado quando o banco de dados desse usuário vai ser apagado e no teste seguinte vamos ter o usuário padrão sem alterações.

Dificuldades

Não tive muitas dificuldades além da falta de experiência com python.

Conclusões

Com muito mais material de apoio disponível para utilizarmos na parte de back-end conseguimos ter uma compreensão melhor, e mais facilidade para trabalhar com o mesmo até o momento não foi realizado nada grande então ainda não tenho muitas conclusões.

Referencias

Videos:

https://www.youtube.com/live/QShMRcicxnE?si=_9J8XERRMZY2XoMz

<https://www.youtube.com/live/-Pi5AmOfL2s?si=oxxmizLxnZpywGzx>

<https://www.youtube.com/live/WnhDgVLYfx0?si=XikLUGj24x18Amtt>

Artigos e material de apoio:

<https://fastapidozero.dunossauro.com/4.0/>

<https://fastapidozero.dunossauro.com/4.0/01/>

<https://fastapidozero.dunossauro.com/4.0/03/>

Também realizei o quiz disponível em cada aula:

https://fastapidozero.dunossauro.com/4.0/quizes/aula_01/

https://fastapidozero.dunossauro.com/4.0/quizes/aula_03/